

Paradigm Shifts in Context Engineering: A Historical and Architectural Analysis of Instruction Optimization in Probabilistic Language Models

The operational interface between human operators and neural language architectures has transitioned from basic text completion to a highly structured engineering discipline¹. Because large language models operate probabilistically rather than deterministically, achieving consistent, high-fidelity outputs requires a systematic control framework to manage the context window¹. Despite rapid advancements in model scale, many practitioners fail to achieve optimal, single-pass instruction execution³. This difficulty stems from a fundamental mismatch between natural language intuition and the underlying mathematical attention mechanisms of transformer-based architectures³.

To establish deterministic control over probabilistic systems, context configuration must be treated as a rigorous engineering contract². This analysis traces the historical development of context configuration, examines the architectural limitations that cause instruction failure, and presents a standardized framework for designing contexts that achieve reliable execution in a single turn².

The Chronological Evolution of Context Configuration

The evolution of generative model control can be divided into distinct developmental phases¹. These phases track the transition of language models from simple text-prediction engines to active, tool-using agentic systems¹.

Pre-2010 to 2021: Syntactic Alignment and In-Context Optimization

The earliest computational foundations began in the pre-2010 era with statistical language models and n-gram architectures, which established the concept of guiding predictive distributions through historical context⁷. In 2015, the introduction of the self-attention mechanism fundamentally changed context tracking, providing models with the capacity to establish long-range dependencies and dynamic focus⁸. By 2018 and 2019, post-training developments turned toward template-based prompt generation, control codes, and task-specific fine-tuning to constrain text outputs⁸.

The release of GPT-2 in 2019 established the zero-shot era, where models were prompted to complete natural language patterns without gradient updates¹. In 2020, GPT-3 introduced few-shot prompting, proving that models possess latent in-context learning capabilities¹. However, early in-context learning was highly sensitive to formatting choices, the quantity of exemplars, and class distributions¹. Changes in the sequence order or classification balance of

examples often degraded accuracy by up to 40%¹⁰.

2022 to 2024: Sequential Logic Topologies and Standardized Taxonomies

In January 2022, Google Research demonstrated that prompting a 540-billion parameter model to generate sequential logic steps (commonly referred to as CoT prompting) unlocked latent deductive capacities¹. This simple technique allowed the model to outperform fine-tuned baselines on complex mathematical benchmarks like GSM8K using only eight in-context exemplars¹.

This milestone shifted the field from zero-shot execution to non-linear logical topologies¹:

- **Reason and Act (ReAct)**: Integrated step-by-step cognitive steps with real-time external tool actions¹.
- **Tree of Thoughts (ToT)**: Introduced dynamic tree search, allowing the system to generate multiple logical paths, evaluate intermediate progress, and backtrack when a path failed¹.
- **Graph of Thoughts (GoT)**: Modeled information as an arbitrary graph where semantic units of task resolution act as nodes and logical dependencies represent edges¹². This allowed the model to execute parallel evaluation paths, combine multiple distinct logic steps, and run feedback loops¹².

By June 2024, researchers published *The Prompt Report*, which established a standardized vocabulary of 33 terms and a systematic taxonomy of 58 distinct text-based prompting techniques, marking the transition of the field into a mature academic discipline¹.

2025 to 2026: Agentic Systems, Structured Enforcements, and Minimalist logic

By 2025, the release of the Model Context Protocol (MCP) standardized how language models interface with external data sources, enterprise tools, and multi-agent workflows¹. Prompt design evolved from simple text instructions to specifying dynamic node behaviors and state transitions in agentic graphs¹.

At the same time, native API integrations began enforcing schema-first structured outputs¹. Rather than attempting to parse unstructured natural language outputs, developers used prompts to define strict JSON schemas¹. By late 2025, the adoption of structured output

interfaces in production applications grew to 73%, up from 31% in early 2024¹.

This shift also drove the adoption of minimalist logic tracking, such as the Chain of Draft (CoD) paradigm¹⁷. While traditional CoT prompting achieved high accuracy, its verbose step-by-step explanations generated significant token overhead and latency¹⁷. Developed by researchers at Zoom Communications, Chain of Draft instructs models to generate highly compressed intermediate steps capped at a strict word limit (such as five words per step)¹⁷. This approach

matches the accuracy of verbose step-by-step methods while using as little as **7.6%** of the tokens, substantially lowering computational costs and response latency¹⁷.

Era	Primary Control Strategy	Technical Mechanism	Key Benchmark Performance / Metric
Statistical Foundations (Pre-2010) ⁷	N-grams and statistical sequences	Direct sequence matching	Foundational probability modeling ⁷
Attention Revolution (2015) ⁸	Self-attention matrices	Dynamic sequence weights	Unlocked context-aware translation ⁸
Zero-Shot Paradigm (2019) ¹	Direct natural language completion	Causal sequence continuation	High performance variance ¹
Few-Shot Breakthrough (2020) ¹	In-Context Learning (ICL)	Exemplar sequence prepending	Highly sensitive to example ordering ¹⁰
Deductive Sequences (2022) ¹	Step-by-step logic (CoT)	Sequence-based step execution	Surpassed fine-tuned baselines on GSM8K ¹
Logical Topologies (2023–2024) ¹	Trees and arbitrary graphs of thoughts	Backtracking and parallel path-merging	62% improvement in sorting over baseline methods ¹⁴
Systemic Schemas (2025–2026) ¹	Schema-first structured formats	Native JSON validation and MCP tools	Structured outputs used in 73% of production apps ¹

Architectural Constraints and the Anatomy of Prompt Failures

Understanding why natural language inputs fail in production requires analyzing the structural

attention constraints of transformer architectures³. Many practitioners treat models like human conversational partners, which introduces several common failure modes³.

The "Lost-in-the-Middle" Phenomenon

A major challenge in context design is the position bias of transformer-based models²². Extensive evaluation shows that models exhibit a U-shaped accuracy curve when retrieving information from long contexts²⁴. Retrieval performance is highest when target information is placed at the absolute beginning or end of the context window, and degrades significantly when relevant information is positioned in the middle²⁴.



In 2025, researchers identified the mathematical and structural causes of this positional bias²²:

1. **Causal Attention Masking:** Unidirectional, decoder-only architectures restrict tokens to only attending to preceding tokens to maintain left-to-right generation²². This design means tokens at the very beginning of the prompt are visible to and attended to by every subsequent token in the entire context window²³. In contrast, a token in the middle of the sequence is only visible to the tokens that follow it, reducing its mathematical impact on the model's final hidden states³.
2. **Attention Sinks:** During pre-training, initial sequence boundary tokens consistently absorb a disproportionate share of attention weight across multiple layers, regardless of their semantic content²². This dynamic heavily weights the start of the context window³.
3. **Recency Bias:** Causal sequence generation naturally prioritizes the local context immediately preceding the generation trigger³. Instructions positioned just before the output prompt exert a stronger influence on the model's next-token probabilities than instructions buried further up in the context window³.

Information Dilution and the Softmax Attention Bottleneck

Transformer attention calculations utilize a *Softmax* function to distribute attention weights across the entire input sequence³:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

When a prompt is padded with irrelevant details—known as "context stuffing"—the signal-to-noise ratio (SNR) is significantly reduced³. Because the *Softmax* calculation must distribute attention weights across a larger total number of tokens, the mathematical weight assigned to critical instructions decreases³. This attention dilution increases the risk of hallucination and leads to instruction-following failures³.

Failure Pattern	Architectural Cause	Operational Consequence	Corrective Engineering Action
Lost-in-the-Middle [cite: 3, 26]	Causal attention masking and attention sinks ³	Model ignores or misses rules placed in the middle of long prompts ³	Place core constraints and task definitions at the absolute beginning and end ³ .
Expert Persona Mediocrity [cite: 3]	Broad semantic activation in latent space	Generates generic, formulaic responses ³	Define personas using: <i>Domain + Experience Signal + Behavioral Note</i> ³ .
Instruction/Data Confusion [cite: 2, 3]	Unstructured sequence processing	Data is mistakenly executed as instructions ²	Isolate context and raw data inputs using explicit XML delimiters ² .
Vague Constraints [cite: 3]	Missing mathematical evaluation triggers	Model guesses the acceptable boundaries ³	Replace qualitative descriptors with clear binary constraints ³ .

Attention Dilution (SNR Drop) [cite: 3]	<i>Softmax</i> weight distribution over irrelevant tokens ³	Spikes in hallucinations and instruction omission ³	Compress inputs, rerank documents, and remove conversational filler ³ .
Multi-Task Cramming [cite: 3, 4]	High task complexity in a single pass	Model fails to execute all requested steps ³	Chain tasks into isolated steps with intermediate evaluation gates ³ .

This structural complexity highlights why prompt engineering has developed a highly specialized labor profile²⁷. A systematic analysis of over 20,000 LinkedIn job postings showed that while prompt engineering roles accounted for less than 0.5% of the sampled listings, they required a unique set of skills combining AI technical knowledge (22.8%), prompt design (18.7%), structured communication (21.9%), and creative problem-solving (15.8%)²⁷.

Required Skill Dimension	Percentage Demand	Practical Application
AI Technical Knowledge [cite: 27]	22.8%	Understanding attention mechanics, context constraints, and token usage ³
Prompt Design [cite: 27]	18.7%	Constructing systematic, reusable templates and dynamic variable layouts ²⁷
Structured Communication [cite: 27]	21.9%	Designing precise, unambiguous technical descriptions ³
Creative Problem-Solving [cite: 27]	15.8%	Debugging output failures, evaluating edge cases, and designing test loops ⁴

Standardized Methodological Paradigms

To address these architectural limitations, researchers have developed specialized prompting techniques that isolate logical execution and optimize attention weight¹.

Decomposition Strategies: Least-to-Most and Program of Thoughts

Decomposition strategies break down complex, multi-step problems into isolated operations to prevent attention drift²⁹:

- **Least-to-Most Prompting (LtM)**: First prompts the model to decompose a complex problem into a sequence of simpler subproblems³⁰. The model then solves these subproblems in order, appending each intermediate solution back into the context window as input for the next step³⁰. On compositional generalization benchmarks like SCAN, Least-to-Most prompting improved execution accuracy from 16% to over 99% , showing a strong ability to generalize to harder problems than those shown in the prompt exemplars³⁰.
- **Program of Thoughts (PoT)**: Offloads mathematical computation and symbolic processing to external systems³³. The prompt directs the model to express its logical steps as executable code (typically Python) and execute it using an external interpreter³⁴. This separation of logical structuring from arithmetic processing yields an average accuracy gain of 12% on complex numerical and financial datasets³³.

Parallelization: Skeleton-of-Thought

To address generation latency, Skeleton-of-Thought (SoT) parallelizes output decoding²⁰. Instead of writing a long answer sequentially, the model is prompted to first generate a concise skeleton outline of the final response³⁷. The system then triggers parallel API calls to expand on each point of the skeleton concurrently, bypassing sequential decoding bottlenecks to improve response speeds³⁷.

Metacognitive Self-Correction

To prevent logical errors from compounding, Metacognitive Prompting (MP) implements a structured, introspective self-evaluation process based on human metacognitive structures³⁸.



— Correct identified errors and finalize the response [cite: 38, 41]

5. Confidence Calibration

— Quantify the probability of accuracy to align certainty with correctness [cite: 41, 42]

In multi-agent environments, this strategy is implemented via Metacognitive Self-Correction (MASC)⁴³. MASC uses unsupervised, step-level error detection to predict sequence consistency⁴³. If an anomaly is flagged, a correction agent is triggered to revise the step before the data flows downstream, preventing cascading logical failures⁴³.

Designing Contexts for Single-Pass Determinism

To consistently get the desired output in a single pass, context design must transition from an ad hoc chat interface to a formal "Prompt-as-a-System" approach³. This method structures the entire context window as a clean interface contract².

The Standardized Context Architecture

This framework organizes information to optimize attention weight distribution, placing core system rules where they receive the highest mathematical attention, and isolating data inside XML tags to prevent instruction-to-data confusion².

===== SYSTEM INSTRUCTION BLOCK

[Redacted]

[system.rules]

[Global behavioral boundaries, output styling, and safety restrictions]

[/system.rules]

System Role

[Domain Expertise] + [Experience Level] + [Operational Boundaries]

Task Instructions

1. [Primary execution instruction]

2. [Sequential operation step]

3. [Explicit negative restriction]

Execution Context

[context]

<document id="1">

[Isolate context chunk or data block]


```

</document>
</context>

## Target Query
<query>
Immediate task to execute
</query>

## Output Response Contract
<response_format>
[strict output schema or target JSON configuration]
</response_format>

===== RUNTIME EXECUTION GATE =====

```

Structural Section	Syntactic Delimiter Paradigm	Architectural Rationale	Functional Contribution
1. System Rules	XML <system_rules> tags ⁵	Places high-priority rules at the top of context to utilize attention sinks ³	Prevents the model from deviating from global constraints and safety boundaries ² .
2. Role Definition	Markdown Header ## System Role [cite: 2, 3]	Narrows the model's active training distribution to professional data spaces ²	Anchors tone and operational constraints, avoiding generic or average outputs ² .
3. Task Instructions	Numbered list under ## Task Instructions [cite: 4, 5]	Enforces a clear order of execution ⁴	Directs step-by-step processing and defines explicit logic pathways ⁴ .
4. Context Isolation	Delimited tags <context> and <document> [cite: 5, 44]	Prevents data text from being processed as active code instructions ²	Completely isolates reference documents, data inputs, and raw

			logs ² .
5. Target Query	Dedicated <query> tags at the end of the context ²	Leverages recency bias to focus attention on the final task ²	Triggers the final operation using immediate execution context ² .
6. Output Response Contract	Standardized tags <response_format> [cite: 2]	Enforces schema-first execution to guarantee parseable outputs ¹	Controls output formatting and schema alignment for downstream parsing ¹ .

Context Engineering Best Practices

To ensure consistent execution in a single turn, context design should apply several tactical techniques:

- **XML Tag Isolation:** Language models are pre-trained heavily on structured code and markup language⁵. Wrapping instructions, background data, and target queries in nested, descriptive XML tags (e.g., <document id="1">) helps the model parse the prompt's logical hierarchy and significantly reduces instruction-following errors⁵.
- **Enforce Binary Constraints:** Avoid using qualitative guidelines like "be concise"³. Instead, specify unambiguous, binary boundaries that the system can evaluate directly (such as "limit responses to three sentences" or "output exactly five bullet points")³.
- **Apply the CARE Framework:** Align context design with the CARE framework: **C**ontext (role, background, industry), **A**ction (specific operational verbs), **R**esult (exact format, schema, layout), and **E**valuation (self-assessment and validation triggers)⁴.
- **Response Prefilling:** For API deployments that support response prefilling, developers can pre-populate the model's initial response sequence (e.g., forcing a JSON response by prefilling { or starting a specific XML response block like <output>)⁵. This enforces structural constraints and prevents formatting deviations⁵.
- **Implement Baseline Testing:** Run five identical baseline tests during prompt design to isolate variations, measure output consistency, and evaluate instruction drift under identical inputs⁴.

The Transition to Programmatic Prompt Generation

Manual prompt optimization is increasingly being automated by systematic meta-prompting platforms⁴⁶:

- **Conductor Orchestration:** Systems use highly capable model architectures (such as Claude 3.5 Sonnet or GPT-4o) as prompt-generation "conductors" to design optimized system instructions for smaller, faster downstream execution models⁴⁶.
- **Programmatic Prompt Compilation:** Frameworks like DSPy automate prompt design⁹.

Instead of manually adjusting phrasing, developers define functional modules with clear inputs and expected outputs⁴⁸. The DSPy compiler then runs optimization loops over a representative dataset, automatically selecting optimal in-context examples, formatting structures, and instruction phrases based on empirical performance scores⁹.

Conclusions

Context engineering has evolved from basic natural language instructions to a rigorous software engineering discipline governed by attention mechanics and positional biases¹. Achieving consistent, single-pass instruction execution requires treating the prompt window as a systematic, structured interface contract².

To build reliable human-AI systems, organizations should adopt the following core principles:

- **Transition from Casual Prompting to Context Engineering:** Avoid sending raw, unformatted text blocks to language models³. Isolate different parts of the prompt using XML delimiters, apply clear markdown formatting, and place primary instructions at the beginning and end of the context window to prevent them from being lost in the middle³.
- **Enforce Schema-First Constraints:** Standardize on structured output schemas (such as JSON mode or function calling) for production integrations to guarantee parseable outputs¹.
- **Use Minimalist and Decomposed Logics:** For logical operations, use decomposition techniques like Least-to-Most prompting to break down complex tasks³⁰, or apply minimalistic paradigms like Chain of Draft to reduce token overhead and latency¹⁷.
- **Implement Programmatic Optimization:** Integrate meta-prompting and compiler frameworks like DSPy to automate prompt generation, benchmark output variations, and evaluate instruction-following performance over representative datasets⁹. Ensure prompt configurations are version-controlled alongside application code⁴.

Works cited

1. What Is Prompt Engineering? - HeyNeo, <https://heyneo.com/blog/what-is-prompt-engineering>
2. Prompt Engineering Across the OpenAI, Anthropic, and Gemini APIs | Steve Kinney, <https://stevekinney.com/writing/prompt-engineering-frontier-llms>
3. Why your prompts fail: The "Lost in the Middle" effect and 6 other structural mistakes (with fixes) - Reddit, https://www.reddit.com/r/PromptEngineering/comments/1sttvqe/why_your_prompts_fail_the_lost_in_the_middle/
4. Prompt Engineering 101: How to Write Better Prompts for AI Tools - Empathy First Media, <https://empathyfirstmedia.com/prompt-engineering-101-how-to-write-better-prompts-for-ai-tools/>
5. How to Write Better Claude Prompts: The Official Framework from Anthropic's Engineers, <https://tribeacademy.sg/blog/claude-prompts/>
6. n8n + Claude AI Agent for Odoo Automation in 30 Minutes - Byte Legions,

- <https://bytelegions.com/build-ai-business-agent-n8n-claude-30-minutes/>
7. A Brief History of Prompt: Leveraging Language Models. - arXiv, <https://arxiv.org/html/2310.04438v1>
 8. [2310.04438] A Brief History of Prompt: Leveraging Language Models. (Through Advanced Prompting) - arXiv, <https://arxiv.org/abs/2310.04438>
 9. The Prompt Report: A Systematic Survey of Prompting Techniques - Sander Schulhoff, https://trigaten.github.io/Prompt_Survey_Site/
 10. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques - Ask Rally, <https://askrally.com/paper/the-prompt-report-a-systematic-survey-of-prompt-engineering-techniques>
 11. Demystifying Chains, Trees, and Graphs of Thoughts - arXiv, <https://arxiv.org/html/2401.14295v3>
 12. Graph of Thoughts — A Graph-Based Reasoning Framework for LLMs, https://systems-analysis.ru/eng/Graph_of_Thoughts
 13. Demystifying Chains, Trees, and Graphs of Thoughts - IEEE Computer Society, <https://www.computer.org/csdl/journal/tp/2025/12/11123142/297Ky5e7xAY>
 14. [2308.09687] Graph of Thoughts: Solving Elaborate Problems with Large Language Models, <https://arxiv.org/abs/2308.09687>
 15. The Prompt Report: A Systematic Survey of Prompting Techniques - BibBase, <https://bibbase.org/network/publication/schulhoff-ilie-balepur-kahadze-liu-si-li-gupta-et-al-thepromptreportasystematicsurveyofpromptingtechniques-2024>
 16. Free Claude Prompt Engineering Course — Master Anthropic's AI with XML Tags, Extended Thinking & MCP | FreeAcademy, <https://freeacademy.ai/courses/prompt-engineering-claude>
 17. Chain of Draft: Thinking Faster by Writing Less - arXiv, <https://arxiv.org/html/2502.18600v1>
 18. (PDF) Chain of Draft: Thinking Faster by Writing Less - ResearchGate, https://www.researchgate.net/publication/389391697_Chain_of_Draft_Thinking_Faster_by_Writing_Less
 19. What is Chain of Draft (CoD)? Faster LLM Reasoning Explained - F22 Labs, <https://www.f22labs.com/blogs/chain-of-draft-thinking-faster-by-writing-less/>
 20. arXiv:2502.18600v2 [cs.CL] 3 Mar 2025, <https://arxiv.org/pdf/2502.18600>
 21. The Prompt Report Part 1: A Systematic Survey of Prompting Techniques, <https://ghost.oxen.ai/the-prompt-report-part-1-a-systematic-survey-of-prompting-techniques/>
 22. LOST IN THE MIDDLE: AN EMERGENT PROPERTY FROM INFORMATION RETRIEVAL DEMANDS IN LLMS - OpenReview, <https://openreview.net/pdf?id=XSHP62BCXN>
 23. The 'Lost in the Middle' Problem — Why LLMs Ignore the Middle of Your Context Window, https://dev.to/thousand_miles_ai/the-lost-in-the-middle-problem-why-llms-ignore-the-middle-of-your-context-window-3al2
 24. Lost in the Middle: How Language Models Use Long Contexts - ResearchGate, https://www.researchgate.net/publication/378284067_Lost_in_the_Middle_How_Language_Models_Use_Long_Contexts

25. Your AI Agent Forgets Everything? You're Solving the Wrong Problem. | by Can Demir,
<https://medium.com/@candemir13/your-ai-agent-forgets-everything-youre-solving-the-wrong-problem-648466172447>
26. Lost-in-the-Middle Effect | LLM Knowledge Base - Promptmetheus,
<https://promptmetheus.com/resources/llm-knowledge-base/lost-in-the-middle-effect>
27. Prompt Engineer: Analyzing Hard and Soft Skill Requirements in the AI Job Market - arXiv, <https://arxiv.org/html/2506.00058v2>
28. Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review - arXiv, <https://arxiv.org/html/2310.14735v5>
29. The Prompt Report: A Systematic Survey of Prompting Techniques - arXiv, <https://arxiv.org/html/2406.06608v1>
30. LEAST-TO-MOST PROMPTING ENABLES COMPLEX REASONING IN LARGE LANGUAGE MODELS - OpenReview,
<https://openreview.net/references/pdf?id=b93l8WgU8>
31. Least-to-Most Prompting,
https://learnprompting.org/docs/intermediate/least_to_most
32. LEAST-TO-MOST PROMPTING ENABLES COMPLEX REASONING IN LARGE LANGUAGE MODELS - Department of Computing Science,
<https://webdocs.cs.ualberta.ca/~daes/papers/iclr23a.pdf>
33. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks - ResearchGate,
https://www.researchgate.net/publication/365701664_Program_of_Thoughts_Prompting_Disentangling_Computation_from_Reasoning_for_Numerical_Reasoning_Tasks
34. Program-of-Thoughts Strategy - Emergent Mind,
<https://www.emergentmind.com/topics/program-of-thoughts-pot-strategy>
35. Scaling Code-Assisted Chain-of-Thoughts and Instructions for Model Reasoning,
<https://neurips.cc/virtual/2025/poster/117205>
36. TIGER-AI-Lab/Program-of-Thoughts: Data and Code for Program of Thoughts [TMLR 2023] - GitHub, <https://github.com/TIGER-AI-Lab/Program-of-Thoughts>
37. Skeleton-of-Thought: Prompting LLMs for Efficient Parallel Generation - arXiv, <https://arxiv.org/html/2307.15337v3>
38. arXiv:2308.05342v1 [cs.CL] 10 Aug 2023, <https://arxiv.org/pdf/2308.05342>
39. Metacognitive Prompting Improves Understanding in Large Language Models - arXiv, <https://arxiv.org/html/2308.05342v4>
40. Metacognitive Self-Correction for Multi-Agent System via Prototype-Guided Next-Execution Reconstruction - arXiv, <https://arxiv.org/html/2510.14319v1>
41. Prompting best practices - Claude API Docs,
<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>
42. Claude System Prompt Explained: What's Inside and Why It Matters - Tactiq,
<https://tactiq.io/learn/claude-system-prompt>
43. Meta-Prompting: LLMs Crafting & Enhancing Their Own Prompts - IntuitionLabs,

<https://intuitionlabs.ai/pdfs/meta-prompting-llms-crafting-enhancing-their-own-prompts.pdf>

- 44. Meta-Prompting: LLMs Crafting & Enhancing Their Own Prompts | IntuitionLabs,
<https://intuitionlabs.ai/articles/meta-prompting-llm-self-optimization>
- 45. A Complete Guide to Meta Prompting - PromptHub,
<https://www.prompthub.us/blog/a-complete-guide-to-meta-prompting>